

Lab 6

Lab 5 gave the game a visible world — a grass terrain built from Tiled map data. But the "player" was still just an invisible observer looking at a static grid. Lab 6 introduces a controllable player character, a camera that follows them, and coordinate system transformations that tie everything together. Clicking to spawn bombs now works correctly in world space rather than screen space.

PlayerObject — a movable character

A new `Models/PlayerObject.cs` class represents the player. Unlike `AnimatedGameObject`, it does not inherit from `RenderableGameObject` — it manages its own texture and position directly:

```
public class PlayerObject : GameObject
{
    public int X { get; set; } = 100;
    public int Y { get; set; } = 100;

    private Rectangle<int> _source = new(0, 0, 48, 48);
    private Rectangle<int> _target = new(0, 0, 48, 48);

    private readonly int _textureId;

    private const int Speed = 128; // pixels per second

    public PlayerObject(int id) : base(id)
    {
        _textureId = GameRenderer.LoadTexture(Path.Combine("Assets",
"player.png"), out _);
        if (_textureId < 0)
        {
            throw new Exception("Failed to load player texture");
        }

        UpdateTarget();
    }
}
```

The player starts at world position `(100, 100)` and uses a 48×48 pixel sprite loaded from `Assets/player.png`. The `_source` rectangle selects the entire texture (no sprite sheet animation here — just a single frame). The `_target` rectangle determines where the player is drawn in world space.

Movement is time-based, using the same pattern established in Lab 4 for animations. The speed is defined as 128 pixels per second, and the actual displacement is scaled by the elapsed time:

```
public void UpdatePosition(double up, double down, double left, double right,
int time)
{
    var pixelsToMove = Speed * (time / 1000.0);
```

```

        Y -= (int)(pixelsToMove * up);
        Y += (int)(pixelsToMove * down);
        X -= (int)(pixelsToMove * left);
        X += (int)(pixelsToMove * right);

        UpdateTarget();
    }

```

Each direction parameter is either `0.0` or `1.0`, acting as a multiplier. This means diagonal movement is possible (and slightly faster than cardinal movement — a classic game dev shortcut). After updating the position, `UpdateTarget` recalculates the destination rectangle:

```

private void UpdateTarget()
{
    _target = new(X + 24, Y - 42, 48, 48);
}

```

The offset `(+24, -42)` positions the sprite so that the player's logical position `(X, Y)` corresponds roughly to their feet rather than the top-left corner of the texture. This is a common convention — it makes it easier to reason about where the player "is" relative to the ground.

The player renders itself through the same `RenderTexture` method that terrain uses:

```

public void Render(GameRenderer renderer)
{
    renderer.RenderTexture(_textureId, _source, _target);
}

```

GameCamera — world-to-screen coordinate transformations

The new `GameCamera.cs` class handles the translation between world coordinates (where things exist in the game map) and screen coordinates (where things appear on the monitor):

```

public class GameCamera
{
    public int X { get; set; }
    public int Y { get; set; }

    public int Width { get; set; }
    public int Height { get; set; }

    public Rectangle<int> ToScreenCoordinates(Rectangle<int> rect)
    {
        return rect.GetTranslated(new Vector2D<int>(Width / 2 - X, Height / 2 - Y));
    }
}

```

```

public Vector2D<int> ToWorldCoordinates(Vector2D<int> point)
{
    return point - new Vector2D<int>(Width / 2 - X, Height / 2 - Y);
}

```

The camera has a position (X , Y) in world space and dimensions ($Width$, $Height$) matching the window size. The two methods are inverses of each other:

- `ToScreenCoordinates` takes a rectangle in world space and shifts it so that whatever is at the camera's position ends up centered on screen. The formula $Width / 2 - X$ means: "shift everything left by the camera's X , then right by half the screen width." The result is that the camera position maps to the center of the window.
- `ToWorldCoordinates` does the reverse — given a pixel position on screen (like a mouse click), it computes the corresponding world position.

This is a simple translation-only camera — no zoom, no rotation. But it is enough to make the world feel larger than the window.

GameRenderer — integrating the camera

The renderer creates and owns the camera instance:

```

private readonly GameCamera _camera = new();

```

During construction, it initializes the camera dimensions from the window size:

```

_camera.X = 0;
_camera.Y = 0;

var windowSize = gameWindow.Size;
_camera.Width = windowSize.Width;
_camera.Height = windowSize.Height;

```

This requires a new `Size` property on `GameWindow`, which queries SDL for the current window dimensions:

```

public (int Width, int Height) Size
{
    get
    {
        int width = 0;
        int height = 0;
        _sdl.GetWindowSize((Window*)_window, ref width, ref height);

        return (width, height);
    }
}

```

```
}  
}
```

Every frame, before clearing the screen and drawing anything, the renderer updates the camera to follow the player:

```
var playerPos = _gameLogic.GetPlayerPosition();  
_camera.X = playerPos.X;  
_camera.Y = playerPos.Y;
```

This means the player is always at the center of the screen, and the world scrolls around them.

Both rendering methods now pass their destination rectangles through the camera transformation.

`RenderGameObject` (used for animated objects like bombs) translates the destination:

```
var textureDest = _camera.ToScreenCoordinates(gameObject.TextureDestination);
```

And `RenderTexture` (used for terrain tiles and the player) does the same:

```
var translatedDst = _camera.ToScreenCoordinates(dst);  
_sdl.RenderCopy((Renderer*)_renderer, (Texture*)texture, in src, in  
translatedDst);
```

Without this transformation, the terrain would be locked to the top-left corner of the screen and the player would walk off-screen immediately.

A new static method `ToWorldCoordinates` is also added to the renderer, giving other systems (specifically `InputLogic`) access to the camera's inverse transformation:

```
public static (int X, int Y) ToWorldCoordinates(int X, int Y)  
{  
    if (_instance == null)  
    {  
        throw new InvalidOperationException("GameRenderer instance is not  
initialized.");  
    }  
  
    var worldCoords = _instance._camera.ToWorldCoordinates(new(X, Y));  
    return (worldCoords.X, worldCoords.Y);  
}
```

This follows the same singleton pattern established in Lab 4 for `LoadTexture` — it uses `_instance` to access the camera without threading a reference through every caller.

InputLogic — keyboard movement and coordinate correction

`InputLogic` undergoes its most significant changes. It gains its own frame timing:

```
private DateTimeOffset _lastUpdate = DateTimeOffset.Now;
```

After draining the event queue, it computes how many milliseconds have elapsed since the last frame:

```
var timeSinceLastFrame =  
(int)currentTime.Subtract(_lastUpdate).TotalMilliseconds;  
_lastUpdate = currentTime;
```

Then it samples the keyboard state for the arrow keys and converts them into direction values:

```
var up = 0.0;  
var down = 0.0;  
var left = 0.0;  
var right = 0.0;  
  
if (keyboardState[(int)KeyCode.Up] == 1)  
{  
    up = 1.0;  
}  
  
if (keyboardState[(int)KeyCode.Down] == 1)  
{  
    down = 1.0;  
}  
  
if (keyboardState[(int)KeyCode.Left] == 1)  
{  
    left = 1.0;  
}  
  
if (keyboardState[(int)KeyCode.Right] == 1)  
{  
    right = 1.0;  
}  
  
_gameLogic.UpdatePlayerPosition(up, down, left, right, timeSinceLastFrame);
```

Using `double` values (`0.0` or `1.0`) instead of booleans is a design choice that anticipates analog input — a gamepad stick could provide fractional values for slower movement. For now, it works as a simple on/off toggle.

The bomb spawning logic now converts screen coordinates to world coordinates before placing the bomb:

```

if (mouseButtonStates[(byte)MouseButton.Primary] == 1)
{
    var worldCoords = GameRenderer.ToWorldCoordinates(mouseX, mouseY);
    _gameLogic.AddBomb(worldCoords.X, worldCoords.Y);
}

```

Without this change, clicking in the top-left corner of the screen would always place a bomb at roughly (0, 0) in world space, regardless of where the camera was looking. With the transformation, the bomb appears at the correct world position under the cursor.

A subtle rename also happens: `_keyboardState` (with the underscore prefix, which usually denotes a field) is corrected to `keyboardState`, since it is a local variable inside `ProcessInput`.

GameLogic — wiring the player into the game loop

`GameLogic` creates the player during initialization and exposes two new methods:

```

private PlayerObject? _player;

public void InitializeGame()
{
    _player = new(1000);
    // ... level loading unchanged
}

```

Position updates are forwarded from input:

```

public void UpdatePlayerPosition(double up, double down, double left, double
right, int timeSinceLastUpdateInMs)
{
    _player?.UpdatePosition(up, down, left, right, timeSinceLastUpdateInMs);
}

```

The camera queries the player's position each frame:

```

public (int X, int Y) GetPlayerPosition()
{
    if (_player == null)
    {
        return (0, 0);
    }

    return (_player.X, _player.Y);
}

```

The player renders after all other game objects, ensuring it appears on top:

```
_player?.Render(renderer);
```

This call sits at the end of `RenderAllObjects`, after bombs have been drawn — so the player is always visible on top of explosions.

Summary

By the end of Lab 6, the game has a controllable player character that moves with the arrow keys at 128 pixels per second. A camera follows the player, keeping them centered on screen while the grass terrain scrolls underneath. Mouse clicks correctly place bomb explosions in world space thanks to coordinate transformations between screen and world systems. The game now feels like a navigable space rather than a static image with effects.